

SMART WHOLESALER

B2B Wholesale Supply Portal
Weekly Project Logbook — Weeks 1 to 5

Field	Details
Course	BIT3208 — Advanced Web Design
Student	Joseph Muturi
Reg. No.	BSCCS/2024/34208
Lecturer	NYORO
Semester	Year 4 Sem 2
GitHub	https://github.com/Josemuturi/Smart-Wholesaler-
Stack	React 19 · Vite 8 · FastAPI · SQLite · JWT · Python 3.12

Table of Contents

Week 1 — Project Setup & Development Environment.....	3
1.1 Tasks Completed	3
1.2 Technologies Used	3
1.3 Key Code	3
1.4 PHP → Modern Stack Mapping	3
1.5 Screenshots	3
1.6 Learning Outcomes.....	4
Week 2 — Authentication System & Security	5
2.1 Tasks Completed	5
2.2 Technologies Used	5
2.3 Key Code	5
2.4 PHP → Modern Stack Mapping	5
2.5 Screenshots	5
2.6 Learning Outcomes.....	6
Week 3 — UI/UX Design System & Dashboard	8
3.1 Tasks Completed	8
3.2 Design Token Reference	8
3.3 Key Code	8
3.4 PHP → Modern Stack Mapping	8
3.5 Screenshots	8
3.6 Learning Outcomes.....	9
Week 4 — Product Catalog & State Management.....	10
4.1 Tasks Completed	10
4.2 Key Code	10
4.3 Mock Product Data	10
4.4 PHP → Modern Stack Mapping	10
4.5 Screenshots	10
4.6 Learning Outcomes.....	12
Week 5 — Database Design & Backend Integration	13
5.1 Tasks Completed	13
5.2 Backend File Structure.....	13
5.3 Database Schema (5 Tables)	13
5.4 Entity Relationships	13
5.5 API Endpoints	14
5.6 Demo Credentials	14
5.7 Key Code	14
5.8 Screenshots	14
5.9 Learning Outcomes.....	14

Week 1 — Project Setup & Development Environment

1.1 Tasks Completed

- Installed Node.js v20 LTS, Python 3.12, Git, and VS Code
- Bootstrapped React + Vite project; configured folder structure: src/pages/, src/components/, src/context/, src/utlis/
- Installed core dependencies: react-router-dom, eslint, @vitejs/plugin-react
- Created GitHub repository, wrote README.md, pushed initial scaffold

1.2 Technologies Used

Tool	Version	Purpose
Node.js	20 LTS	JavaScript runtime
Vite	8.x	Build tool & dev server (HMR)
React	19.x	UI component library
react-router-dom	7.x	Client-side SPA routing
Git / GitHub	Latest	Version control

1.3 Key Code

Entry point — main.jsx:

```
ReactDOM.createRoot(document.getElementById('root')).render(<React.StrictMode><App /></React.StrictMode>);
```

Vite config — vite.config.js:

```
export default defineConfig({ plugins: [react()], server: { port: 5173 } });
```

1.4 PHP → Modern Stack Mapping

PHP Concept	Smart Wholesaler Equivalent
<?php echo 'Hello'; ?>	JSX: Hello rendered by React
XAMPP / Apache	Vite dev server (npm run dev)
index.php	main.jsx + index.html
include('header.php')	React component: <Header />
\$_GET / \$_POST	React state, URL params, form handlers

1.5 Screenshots

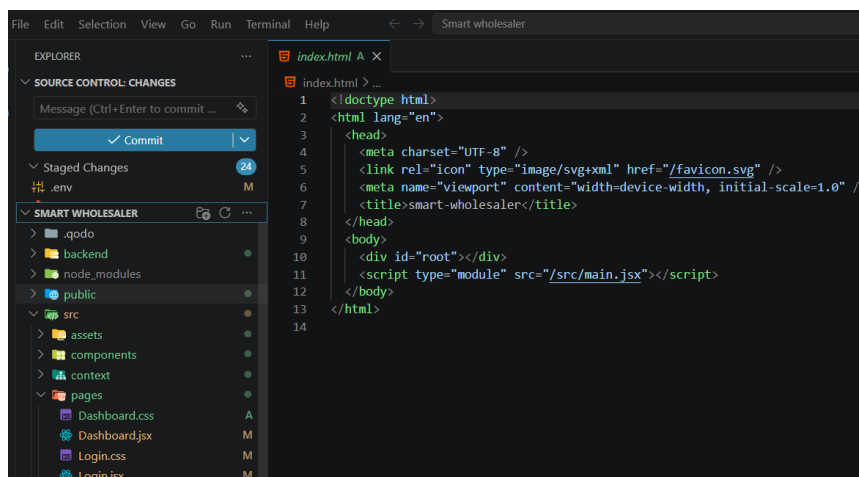


Fig 1.1 — VS Code with project scaffold and staged files ready for first commit

```
PS C:\Smart wholesaler> npm run dev

> smart-wholesaler@0.0.0 dev
> vite

08:09:13 [vite] (client) Re-optimizing dependencies because vite config has changed
Port 3003 is in use, trying another one...

VITE v8.0.13 ready in 847 ms

→ Local: http://localhost:3004/
→ Network: use --host to expose
→ press h + enter to show help
```

Fig 1.2 — Vite dev server running on localhost:3004

```
Problems Output Debug Console Terminal Ports
PS C:\Smart wholesaler> Commit: 31749be - "Week 1: Vite+React scaffold, Hello world page, DB connection test"
>>
```

Fig 1.3 — Week 1 git commit: Vite+React scaffold

1.6 Learning Outcomes

- Understood Vite as a modern replacement for PHP/Apache
- Initialised a React SPA with a structured folder hierarchy
- Practised Git workflow: init → add → commit → push
- Understood the difference between server-rendered PHP and client-side SPA routing

Week 2 — Authentication System & Security

2.1 Tasks Completed

- Created AuthContext.jsx — global auth state (user, login, logout) via React Context API
- Built Login.jsx with email/password form, error handling, loading state, and show/hide password toggle
- Implemented ProtectedRoute.jsx — redirects unauthenticated users; supports role-based access via allowedRoles prop
- Created utils/api.js — centralised HTTP client with JWT Bearer header injection
- Implemented mock auth mode (VITE MOCK_AUTH=true) for offline development
- Tested login redirect flow — preserves intended destination via state.from

2.2 Technologies Used

Technology	Role
React Context API	Global authentication state management
localStorage	JWT persistence across page refreshes
react-router-dom Navigate	Programmatic redirects on login/logout
Fetch API	HTTP requests to FastAPI backend
JWT (JSON Web Token)	Stateless authentication token format

2.3 Key Code

Login function (AuthContext.jsx):

```
const data = await api.post('/auth/login', { email, password });
setToken(data.access_token); setUser(data.user);
```

Route guard (ProtectedRoute.jsx):

```
if (!isLoggedIn) return <Navigate to='/login' state={{ from: location }} replace />;
if (allowedRoles.length && !allowedRoles.includes(user?.role)) return <Navigate to='/dashboard' />;
```

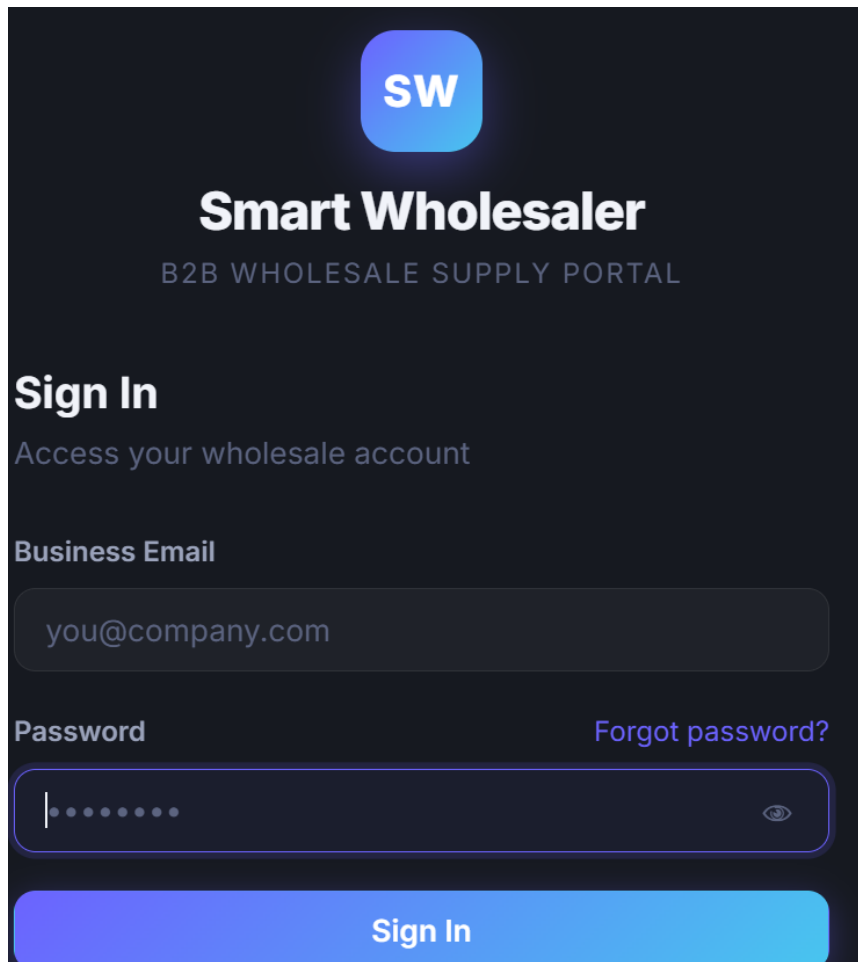
JWT token helpers (api.js):

```
export const getToken = () => localStorage.getItem('secure_duka_token');
headers['Authorization'] = `Bearer ${token}`;
```

2.4 PHP → Modern Stack Mapping

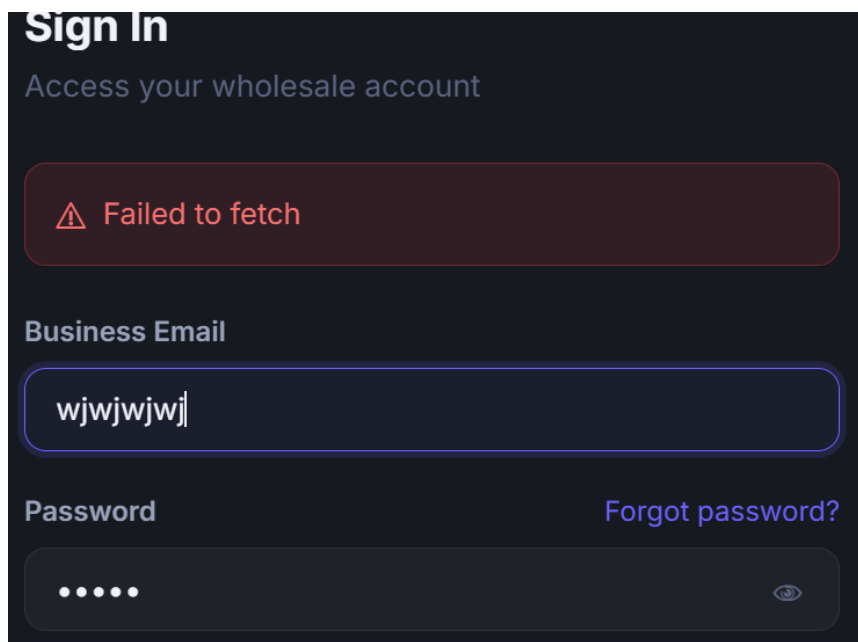
PHP Concept	Smart Wholesaler Equivalent
\$_SESSION['user']	AuthContext user state + localStorage JWT
session_start()	AuthProvider wrapping the component tree
header('Location: /')	navigate('/dashboard', { replace: true })
password_hash() / verify()	bcrypt via passlib on FastAPI backend
mysql_real_escape_string()	Pydantic schema validation on FastAPI

2.5 Screenshots



The image shows the login page for 'Smart Wholesaler', a B2B Wholesale Supply Portal. At the top is a blue rounded square logo with the letters 'SW' in white. Below the logo, the text 'Smart Wholesaler' is displayed in a large, bold, white font, followed by 'B2B WHOLESALE SUPPLY PORTAL' in a smaller, all-caps, white font. The main heading 'Sign In' is in a large, bold, white font, with the subtitle 'Access your wholesale account' in a smaller, lighter white font. Below this, there is a 'Business Email' label followed by a text input field containing 'you@company.com'. Underneath is a 'Password' label followed by a password input field with a show/hide toggle icon (an eye) on the right. To the right of the password field is a link that says 'Forgot password?'. At the bottom is a large, rounded rectangular button with a blue-to-purple gradient, labeled 'Sign In' in white text.

Fig 2.1 — Login page: business email field, password with show/hide toggle, Sign In button



This image shows the same login page as Fig 2.1, but in an error state. A red error message 'Failed to fetch' with a warning triangle icon is displayed in a red box at the top. The 'Business Email' field now contains 'wjwjwjwj'. The 'Password' field is empty, and the 'Forgot password?' link remains visible.

Fig 2.2 — Login error state: 'Failed to fetch' displayed when backend is unreachable

```
<command> [<args>]  
PS C:\Smart wholesaler> git commit -m "Week 2: Design system CSS, wireframe notes, static Login page layout"  
>>
```

Fig 2.3 — Week 2 git commit: design system CSS, wireframe notes, static Login page layout

2.6 Learning Outcomes

- Understood how React Context replaces PHP session globals for state sharing
- Learned the JWT flow: login → token → bearer header → protected resource
- Practised RBAC using a reusable route-guard component
- Understood the difference between stateful PHP sessions and stateless JWT tokens

Week 3 — UI/UX Design System & Dashboard

3.1 Tasks Completed

- Created index.css with a full CSS custom properties design token system; dark mode default, light theme override
- Built ThemeContext.jsx — system-wide dark/light toggle persisted in localStorage
- Built Dashboard.jsx with sidebar, topbar, KPI stat cards, recent orders table, and quick actions
- Added role-aware sidebar navigation (distributor vs. retailer views)
- Implemented skeleton loading animation, mobile hamburger menu, and theme toggle button
- Added Google Fonts (Inter) for premium typography

3.2 Design Token Reference

CSS Variable	Dark Value	Light Value
--color-bg	#0d0f14	#f4f6fb
--color-surface	#161920	ffffff
--color-text-primary	#f1f3f9	#0f1117
--color-accent	#6c63ff	#5b52f0
--color-success	#10b981	#059669
--gradient-primary	#6c63ff → #48c6ef	#5b52f0 → #38b6ff

3.3 Key Code

Theme toggle (ThemeContext.jsx):

```
const toggleTheme = () => setTheme(curr => curr === 'dark' ? 'light' : 'dark');
useEffect(() => { document.documentElement.setAttribute('data-theme', theme);
  localStorage.setItem('sw_theme', theme); }, [theme]);
```

3.4 PHP → Modern Stack Mapping

PHP Concept	Smart Wholesaler Equivalent
CSS in a single .css file	CSS custom properties system in index.css
PHP if/else for dark/light	data-theme attribute + CSS variable overrides
echo \$user['role']	{user?.role === 'distributor' ? ... : ...} in JSX
setcookie('theme', 'dark')	localStorage.setItem('sw_theme', 'dark')

3.5 Screenshots

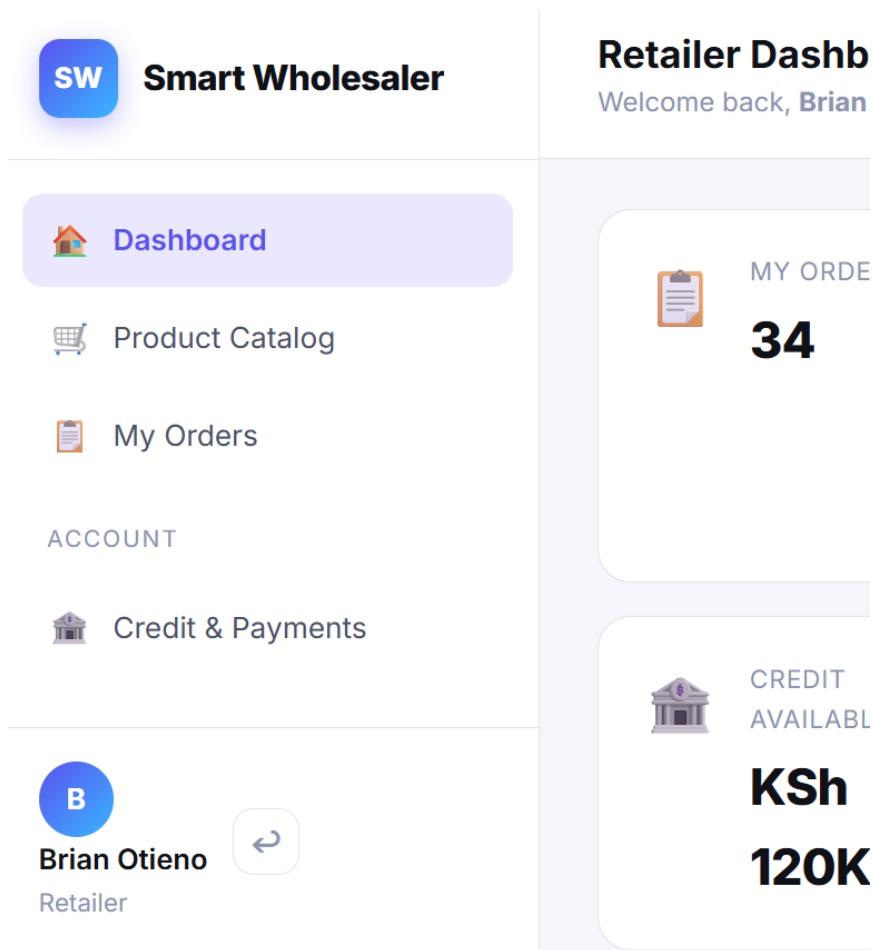


Fig 3.1 — Retailer Dashboard: sidebar navigation, KPI cards (My Orders: 34, Credit: KSh 120K)

```
PS C:\Smart wholesaler> git add weekly-submissions/week3/ src/utlis/api.js src/context/AuthContext.jsx
src/components/ProtectedRoute.jsx
>> git commit -m "Week 3: Form validation, password toggle, api.js utility, AuthContext"
>>
warning: in the working copy of 'weekly-submissions/week3/README.md', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'weekly-submissions/week3/src/pages/Login.jsx', LF will be replaced by
CRLF the next time Git touches it
warning: in the working copy of 'weekly-submissions/week3/src/utlis/api.js', LF will be replaced by CR
LF the next time Git touches it
[main f27df63] Week 3: Form validation, password toggle, api.js utility, AuthContext
```

Fig 3.2 — Week 3 git commit: form validation, password toggle, api.js utility, AuthContext

3.6 Learning Outcomes

- Mastered CSS custom properties as a design token system
- Implemented dark/light theming via a single data attribute on <html>
- Understood React Context as a global state container equivalent to PHP superglobals
- Built complex responsive layouts with CSS Grid and Flexbox

Week 4 — Product Catalog & State Management

4.1 Tasks Completed

- Built ProductCatalog.jsx with responsive product card grid
- Real-time search by name and SKU; category filter chips; multi-criteria sort (Name, Price, Stock)
- Built ProductCard sub-component with quantity stepper respecting min_order_qty
- Implemented cart state (cartCount) with running total in topbar
- Out-of-stock handling: grey overlay, disabled button; skeleton loading; empty state; error state with Retry
- Formatted all prices as Kenyan Shillings using Intl.NumberFormat

4.2 Key Code

Client-side filter & sort:

```
const visible = products
  .filter(p => (cat==='All' || p.category===cat) &&
    p.name.toLowerCase().includes(q))
  .sort((a,b) => sort==='price-asc' ? a.unit_price-b.unit_price :
    a.name.localeCompare(b.name));
```

Add to cart handler:

```
const handleAddToCart = async (product, qty) => { setCartCount(c => c + qty); };
```

4.3 Mock Product Data

SKU	Product	Category	Price (KSh)	Stock
SKU-001	Unga Pembe Wheat Flour	Flour & Grains	2,400	340
SKU-002	Kabras Sugar	Sugar & Sweeteners	2,100	220
SKU-003	Bidco Cooking Oil	Cooking Oils	3,800	95
SKU-004	Maize Meal (Posho)	Flour & Grains	1,350	410
SKU-005	Brookside Fresh Milk	Dairy & Eggs	1,440	60
SKU-006	Ketepa Tea Bags	Beverages	4,500	130
SKU-007	Iodized Table Salt	Sugar & Sweeteners	1,800	0 (OOS)
SKU-008	Sunflower Oil (Pwani)	Cooking Oils	2,100	180

4.4 PHP → Modern Stack Mapping

PHP Concept	Smart Wholesaler Equivalent
SELECT * FROM products	api.get('/products') → state array
WHERE category = ?	products.filter(p => p.category === cat)
ORDER BY price ASC	products.sort((a,b) => a.price - b.price)
\$_SESSION['cart']	cartCount useState + future POST /cart/items
number_format(\$price, 2)	Intl.NumberFormat('en-KE', { currency:'KES' })

4.5 Screenshots

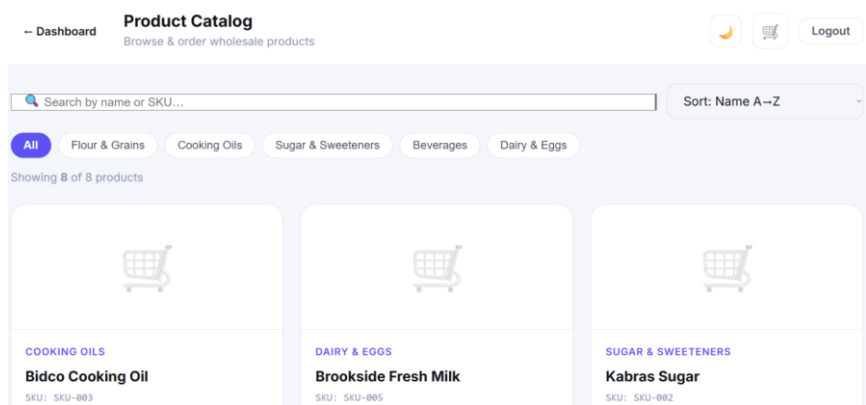


Fig 4.1 — Product Catalog: search bar, category filter chips, sort control, product grid

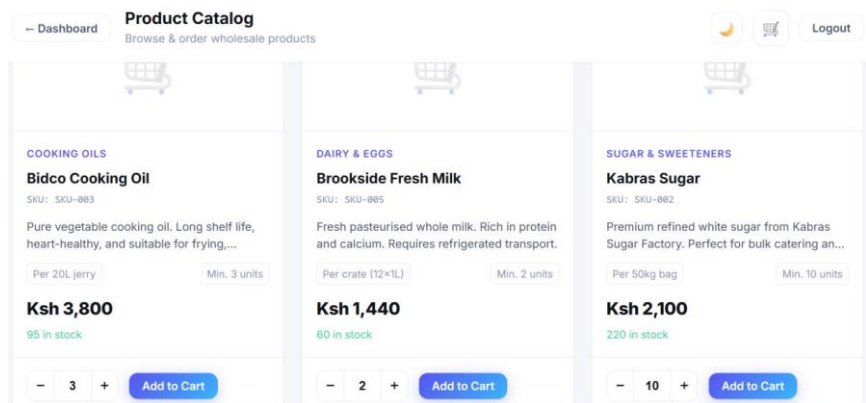


Fig 4.2 — Product cards with descriptions, unit info, price in KSh, quantity stepper, Add to Cart

ORDER ID	PRODUCT	QTY	DATE	TOTAL	STATUS
ORD-001	Unga Pembe 50kg	20	2026-05-18	KSh 48,000	Delivered
ORD-002	Cooking Oil 20L	12	2026-05-19	KSh 21,600	In Transit
ORD-003	Sugar 50kg bags	30	2026-05-20	KSh 63,000	Processing

Fig 4.3 — Recent Orders table on Dashboard: order ID, product, qty, date, total, status badge

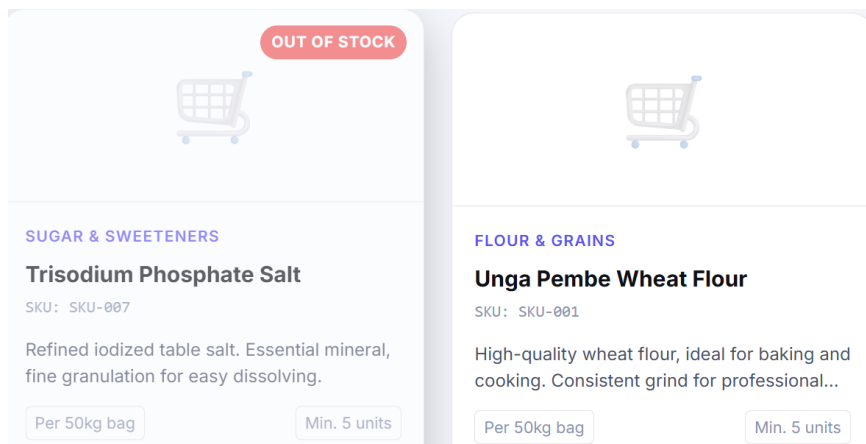


Fig 4.4 — Out of Stock handling: SKU-007 greyed out with 'OUT OF STOCK' badge

```

>> git commit -m "Week 4: Dashboard UI, Product Catalog, dark/light theme toggle"
>> C:\Smart wholesaler>
warning: in the working copy of 'src/pages/Dashboard.jsx', LF will be replaced by CRLF the next time
it touches it
warning: in the working copy of 'src/pages/ProductCatalog.jsx', LF will be replaced by CRLF the next t
ime Git touches it
warning: in the working copy of 'src/context/ThemeContext.jsx', LF will be replaced by CRLF the next t
ime Git touches it
[main 118b5ed] Week 4: Dashboard UI, Product Catalog, dark/light theme toggle
3 files changed, 77 insertions(+), 4 deletions(-)
create mode 100644 src/context/ThemeContext.jsx

```

Fig 4.5 — Week 4 git commit: Dashboard UI, Product Catalog, dark/light theme toggle

4.6 Learning Outcomes

- Managed complex multi-field state: products, categories, search term, sort key, cart count
- Implemented client-side filtering and sorting as a bridge to server-side queries
- Applied UX patterns: skeleton loaders, empty states, error states with retry
- Built reusable sub-components (ProductCard) that receive props and callbacks

Week 5 — Database Design & Backend Integration

5.1 Tasks Completed

- Set up backend/ directory with full FastAPI application structure
- Designed and implemented 5-table SQL schema: users, products, orders, order_items, cart_items
- Built SQLAlchemy ORM models with foreign keys, relationships, and bcrypt password hashing
- Implemented JWT token creation and verification using python-jose
- Created all API endpoints for auth, products, and cart (see 5.4)
- Configured CORS middleware for React dev server (localhost:5173)
- Wrote seed.py — populated 2 demo users and 8 products; set VITE MOCK_AUTH=false
- Tested full login flow end-to-end and verified GET /products returns live database records

5.2 Backend File Structure

File	Purpose
main.py	FastAPI app factory, CORS config, router mounting
database.py	SQLAlchemy engine (SQLite), SessionLocal, get_db()
models.py	ORM models: User, Product, Order, OrderItem, CartItem
schemas.py	Pydantic request/response schemas
auth.py	bcrypt hashing, JWT creation/verification, dependencies
seed.py	Database seeder: 2 users + 8 products
routers/auth.py	POST /auth/login, GET /auth/me
routers/products.py	GET /products, GET /categories, POST /products
routers/cart.py	GET /cart, POST /cart/items, DELETE /cart

5.3 Database Schema (5 Tables)

Table	Key Columns	Notes
users	id, name, email, hashed_password, role, is_active, created_at	role: distributor retailer
products	id, name, sku, category, unit, unit_price, min_order_qty, stock_qty, description	Prices in KSh; SKU unique
orders	id, user_id (FK), status, total_amount, created_at	status: pending processing in-transit delivered cancelled
order_items	id, order_id (FK), product_id (FK), quantity, unit_price	unit_price snapshotted at order time
cart_items	id, user_id (FK), product_id (FK), quantity, added_at	Upsert on duplicate user+product

5.4 Entity Relationships

Relationship	Type	Description
users → orders	One-to-Many	A user can place many orders
orders → order_items	One-to-Many	An order contains many line items

products → order_items	One-to-Many	A product can appear in many order lines
users → cart_items	One-to-Many	A user can have multiple items in their cart
products → cart_items	One-to-Many	A product can be in many users' carts

5.5 API Endpoints

Method	Endpoint	Auth	Description
POST	/auth/login	No	Login with email + password → returns JWT
GET	/auth/me	Yes	Get current user profile
GET	/products	Yes	List all products (supports ?category= & ?search=)
GET	/products/categories	Yes	List all unique categories
POST	/products	Distributor only	Create a new product
GET	/cart	Yes	Get current user's cart
POST	/cart/items	Yes	Add product to cart
DELETE	/cart/items/{id}	Yes	Remove item from cart
DELETE	/cart	Yes	Clear entire cart

5.6 Demo Credentials

Role	Email	Password	Access Level
Distributor	admin@smartwholesaler.com	admin123	Full admin — add products, view all retailers
Retailer	retailer@smartwholesaler.com	retailer123	Browse catalog, add to cart, place orders

5.7 Key Code

FastAPI login endpoint (routers/auth.py):

```
user = db.query(models.User).filter(models.User.email == credentials.email).first()
if not user or not verify_password(credentials.password, user.hashed_password):
    raise HTTPException(status_code=401, detail='Invalid email or password.')
token = create_access_token(data={'sub': user.email})
return { 'access_token': token, 'token_type': 'bearer', 'user': user }
```

SQLAlchemy User model (models.py):

```
class User(Base):
    email = Column(String(150), unique=True, index=True, nullable=False)
    hashed_password = Column(String(255), nullable=False)
    role = Column(String(20), nullable=False, default='retailer')
    orders = relationship('Order', back_populates='user')
```

5.8 Screenshots



Fig 5.1 — Week 5 git commit 467738a: FastAPI backend, SQLite database, JWT auth, logbook

5.9 Learning Outcomes

- Designed a normalised relational database schema with proper foreign keys and indexes

- Learned SQLAlchemy ORM as a modern equivalent to raw mysqli/PDO queries
- Implemented secure bcrypt password hashing (vs. insecure MD5 in legacy PHP)
- Understood stateless JWT authentication vs. stateful PHP sessions
- Built and tested a REST API with FastAPI including CORS configuration for SPA integration
- Practised the full-stack integration loop: DB → ORM → API endpoint → React fetch → UI state